

Question 1-4

4 marks

Indicate whether the following statements are TRUE or FALSE.

1. An abstract class can ONLY contain abstract methods. **F**
2. An interface can contain both method implementation and declaration. **F**
3. It is a good programming practice to avoid strong coupling among classes which interact **T**
4. Which of the following **is** or **are** true about the factory pattern? **C**
 - A. The concrete classes can inherit a superclass
 - B. The concrete classes can implement an interface
 - C. Both A and B
 - D. None of the above

Question 5

4 marks

Class A inherits an abstract class B. In each of the following, indicate whether the statement is valid or invalid. Explain your answer.

1. B b=new B(); (2)
Invalid; cannot create object of an abstract class
2. B b= new A(); (2)
Valid; can assign object of subclass A to variable of abstract class B

Question 6

4 marks

Class C implements interface D. In each of the following, indicate whether the statement is valid or invalid. Explain your answer.

1. D d=new D(); **Invalid; cannot create object of an instance**
2. D d=new C(); **Valid; can assign an object of a class to an interface it implements**

Question 7

2 marks

Give **one reason** why design patterns are important in software development.

Any correct answer (e.g., software development is complex. Design patterns are templates to solve common problems).

Question 8

3

Match the following examples to the main types of design patterns

(3)

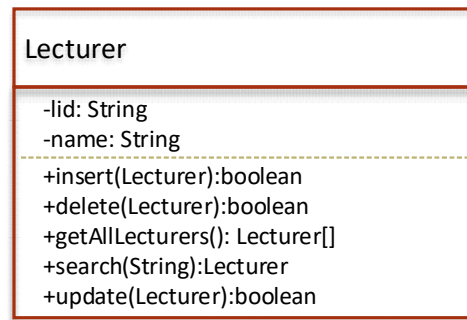
	Type	Example
1	Creational	A Adaptor

- | | | | |
|---|-------------|---|-------------------|
| 2 | Structural | B | DAO |
| 3 | Behavioural | C | Factory |
| 4 | | D | None of the above |

Answer: 1 C; 2 A; 3 D

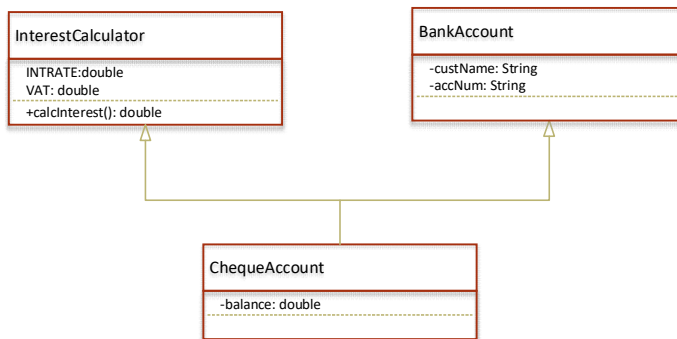
Question 9 6 marks

The diagram shows a class **Lecturer**. The methods **insert**, **delete**, **getAllLecturers**, **search** and **update** are operations on **Lecturer** objects.



- Briefly explain the problem with the class. (2)
 Answer any correct answer (e.g., the **Lecturer** object is tightly tied to the methods that manipulate it. This can make the reduce code re-use). (2 marks)
- Which design pattern can resolve the problem? Briefly explain your answer. (4)
 Answer: **DAO**; this separates the **POCO** from the methods that manipulate it. This way the **POCO** can be re-used in other code.

Question 10 12 marks



The diagram shows **ChequeAccount** inherits **InterestCalculator** and **BankAccount**. **BankAccount** contains **custName** (the customer name) and **accNum** (account number). **InterestCalculator** contains: **INTRATE** (interest rate at 20% per annum) and **VAT** (Vat at 15%) and **calcInterest()** returns the total interest (i.e., the 20% per annum and VAT payable) on the balance. **ChequeAccount** contains the balance (current balance).

- Explain why **InterestCalculator** and not **BankAccount** should be converted to an interface to remove the problem of multiple inheritance. (3)

The class **InterestCalculator** contains constants and the calcInterest() can be declared abstract and implemented by **ChequeAccount**. BankAccount cannot be converted to interface because it contains variables ✓✓✓

2. Write the code for the interface (use the correct access levels) (4)

```
public interface InterestCalculator ✓{  
    public static double INTRATE=0.2; ✓  
    public static double VAT=15.0; ✓  
    public abstract double calcInterest(); ✓  
}
```

3. Complete the code segments for 3.1 and 3.2 below (use the correct access levels):

3.1 Complete the class header (3)

3.2 Implement calcInterest(). (2)

```
public class ChequeAccount ✓ extends Account ✓ implements InterestCalculator ✓{  
    public ChequeAccount (double balance) {  
        super();  
        this.balance = balance;  
    }  
  
    private double balance;  
    public LoanAccount() {  
        // TODO Auto-generated constructor stub  
    }  
    public LoanAccount(String name, String accNum, double balance) {  
        super(name, accNum);  
        this.balance = balance;  
    }  
  
    @Override  
    public double calcInterest() ✓{  
        return (balance*((20.0/100)+(15.0/100))); ✓  
    }  
    public String toString() {  
        return super.toString()+"\nLoanAccount [balance=" + balance + ",  
calcInterest()=" + calcInterest() + "];"  
    }  
}
```